

Analiza i przetwarzanie dźwięku

Projekt 3

Parametryzacja sygnału mowy i klasyfikacja DTW

rozpoznawanie słów oraz identyfikacja mówcy

Autorzy:

Ryszard Czarnecki
Józef Dubois

May 28, 2026

1 Cel projektu

Projekt sprowadza się do dwóch klasycznych zadań przetwarzania mowy: rozpoznawania pojedynczych słów mówionych oraz identyfikacji mówcy na podstawie krótkiej wypowiedzi. W obu używamy własnej parametryzacji sygnału (krótkoterminowe widmo na skali mel) i odległości DTW jako miary podobieństwa między nagraniami. Klasyfikator jest prostym k-NN po DTW, bez żadnej fazy trenowania w sensie ML.

2 Struktura aplikacji

Poza modułami obliczeniowymi projekt zawiera graficzną aplikację desktopową (Rys. 1) zbudowaną w bibliotece Tkinter. Kod podzielony jest na plik główny oraz paczkę `src/`:

- `app.py` punkt wejścia; buduje okno aplikacji i rysuje trzy wykresy: przebieg czasowy sygnału, energię RMS ramek oraz uśrednione widmo FFT.
- `src/basic_functions.py` wszystkie operacje na sygnale: ręczny parser nagłówków RIFF/WAVE (`read_wav`), normalizacja amplitudowa, generowanie okien (`make_window`), trim ciszy, bank filtrów mel i funkcja `extract_features` łącząca pre-emfazę, FFT, log-mel oraz opcjonalne delty i CMN.
- `src/dtw.py` implementacja DTW: budowa macierzy kosztów lokalnych (`local_cost_matrix`) i rekurencja z ograniczeniem Sakoe-Chiba (`dtw`); zwraca odległość, macierz skumulowaną i ścieżkę dopasowania.
- `src/classifier.py` k-NN po DTW (`classify`): oblicza odległość do każdego wzorca z bazy, normalizuje przez $N + M$, głosuje ważone odwrotnością dystansu.
- `src/database.py` ładowanie bazy nagrań (`load_database`): przechodzi przez katalogi `speaker_XX/Znormalizowane/`, kanonizuje nazwy słów (m.in. cyfry $\rightarrow$ polskie odpowiedniki, usunięcie polskich znaków), wyciąga cechy mel i RMS dla każdego pliku.
- `src/evaluation.py` ewaluacja offline: schemat *split per-mówca* (`evaluate_split`) oraz leave-one-out (`evaluate_loo`); drukuje dokładność rozpoznawania słów i identyfikacji mówcy.

3 Baza nagrań i jej kanonizacja

Każdy katalog `speaker_XX` zawiera dwa zestawy WAV-ów: `Nieznormalizowane` i `Znormalizowane`. Korzystamy z tych drugich. Nazwy plików mają format `<słowo>_<numer_próby>.wav`, ale konwencja zapisu *słowa* jest zaskakująco niespójna:

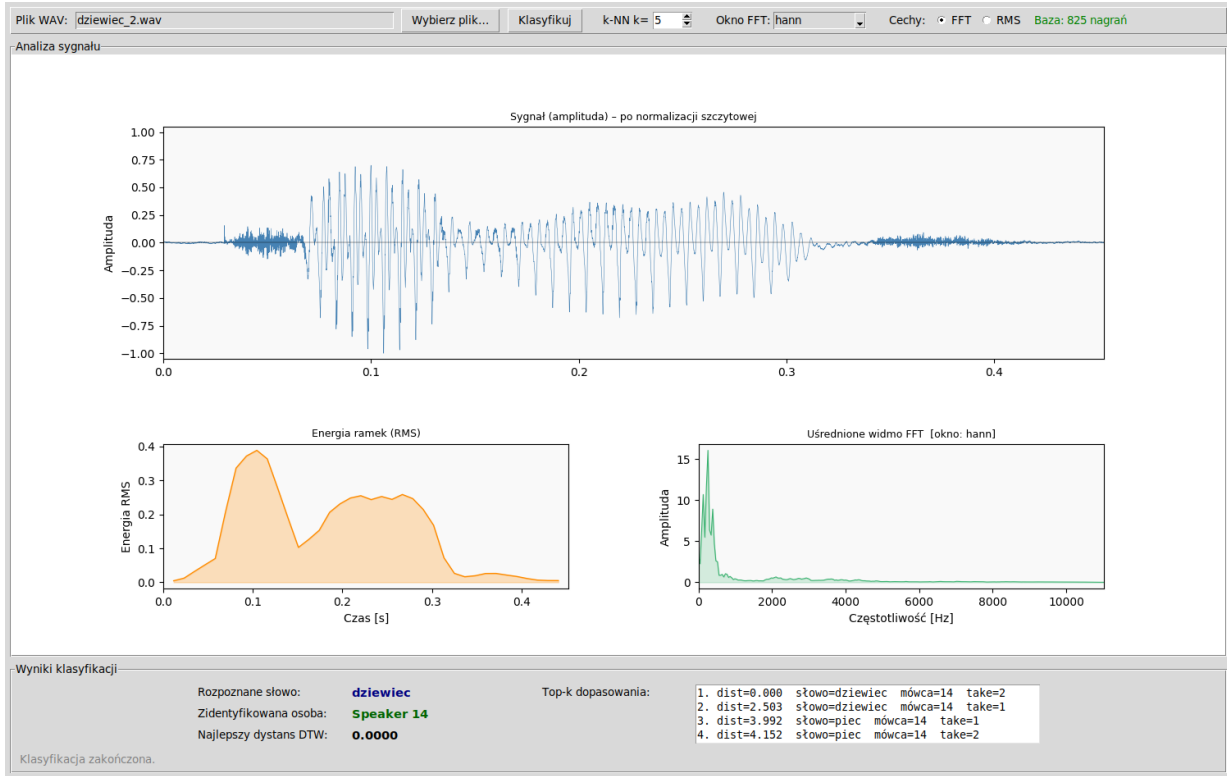
- większość mówców używa polskich słów małymi literami: `kot_1.wav`, `pies_1.wav`, `benzyna_1.wav`;
- `speaker 01` zapisuje cyfry zamiast słów: `0_1.wav`, `10_2.wav`, i to wielkimi literami: `KOT_1.wav`, `ALA MA RUDEGO KOTA_1.wav`;
- polskie znaki czasem są (`samochód`), czasem nie (`samochod`), a u kilku mówców pojawia się dziwny znak (`samochød`, przypuszczalnie pamiątka po jakimś kodowaniu);
- dwóch mówców (07 i 08) numeruje próby od 0 zamiast od 1.

4 Wstępne przetwarzanie sygnału

Każdy plik jest dekodowany ręcznym parserem nagłówków RIFF/WAVE (`read_wav`). Obsługujemy formaty 8/16/24/32 bit PCM, mono i stereo (mieszmamy kanały w mono średnią arytmetyczną). Wyjściowy sygnał to wektor `float64` w zakresie $[-1, 1]$.

Następnie sygnał przechodzi przez trzy kroki przygotowawcze:

(a) Normalizacja amplitudowa. Skalujemy sygnał tak, aby $\max |x[n]| = 1$. To eliminuje różnice głośności pomiędzy nagraniami i jest standardem przy parametryzacji.



Rys. 1: Główne okno aplikacji. Panel górny - przebieg czasowy sygnału po normalizacji szczytowej. Panel dolny - energia RMS ramek (lewy wykres) oraz uśrednione widmo FFT z wybranym oknem (prawy wykres). Sekcja wyników pokazuje rozpoznane słowo, zidentyfikowanego mówcę i listę *top-k* dopasowań DTW z dystansami. Na przykładzie: `dziewiec_2.wav` sklasyfikowany poprawnie jako Speaker 14, dystans DTW = 0,000.

(b) Trim ciszy. Liczymy energię RMS w oknach o długości 512 próbek, hop 256. Próg w decybelach: ramka jest uznana za *niemą*, jeśli jej RMS jest niższy o więcej niż 30 dB od maksymalnej ramki nagrania. Pozostawiamy tylko fragment od pierwszej do ostatniej ramki „głośnej”. W praktyce skraca to nagrania o 20–40% i znacząco poprawia jakość dopasowania DTW. Klasyczne DTW nie ma jak rozróżnić ciszy na początku jednego nagrania od ciszy na początku drugiego i marnuje sporą część ścieżki na niefonetyczne wyrównywanie. Efekt widać na Rys. 2.

(c) Pre-emfaza. Filtr pierwszego rzędu $y[n] = x[n] - 0,97x[n-1]$ wzmacnia wyższe częstotliwości. To kosmetyczna, ale istotna rzecz w analizie mowy. Formanty zwykle siedzą w paśmie 1–4 kHz i bez pre-emfazy są tłumione przez naturalne spadanie widma o ~ 6 dB/oktawę.

5 Parametryzacja: log-energia w pasmach mel

Po preprocessingu sygnał dzielimy na ramki długości $L = 512$ próbek z hopem $H = 256$ (czyli z 50% nakładką). Każdą ramkę mnożymy przez okno Hann’a

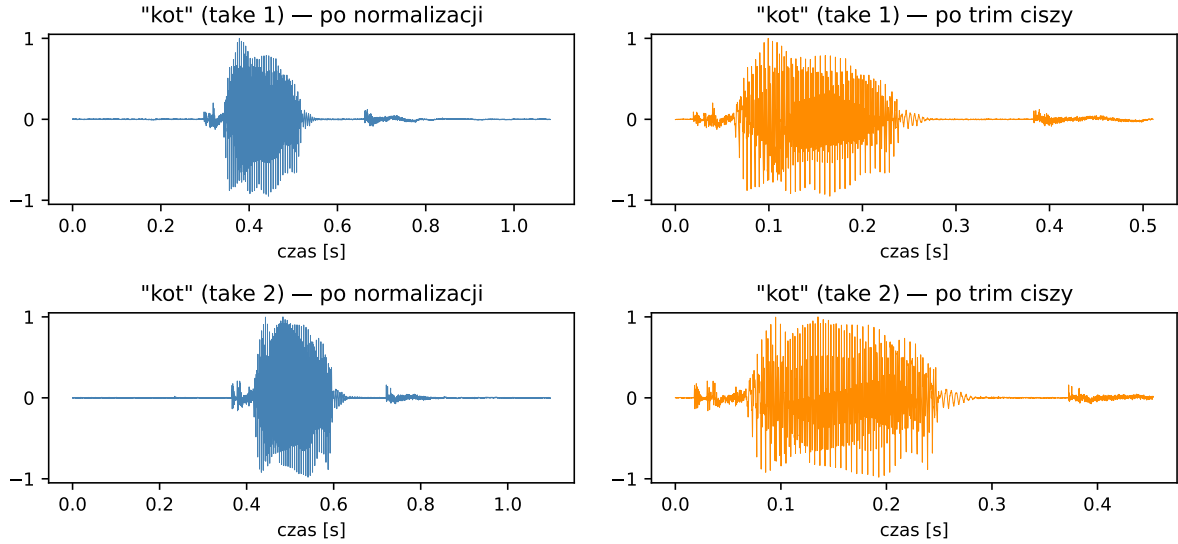
$$w[n] = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{L-1} \right), \quad n = 0, \dots, L-1,$$

liczymy FFT `numpy.fft.rfft` (zwraca $L/2 + 1 = 257$ bin-ów), a następnie *moc widmową* $|X[k]|^2$.

Z 257 binów schodzimy do 24 pasm mel’a. Pasma mel rozłożone są równo na skali percepcyjnej:

$$m(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right).$$

Bank filtrów to 24 trójkątne funkcje wagowe (Rys. 3), rozmieszczone tak, że szczyt i -tego filtra leży w punkcie f_i , a baza filtra rozciąga się do f_{i-1} od dołu i f_{i+1} od góry. Wektor energii w

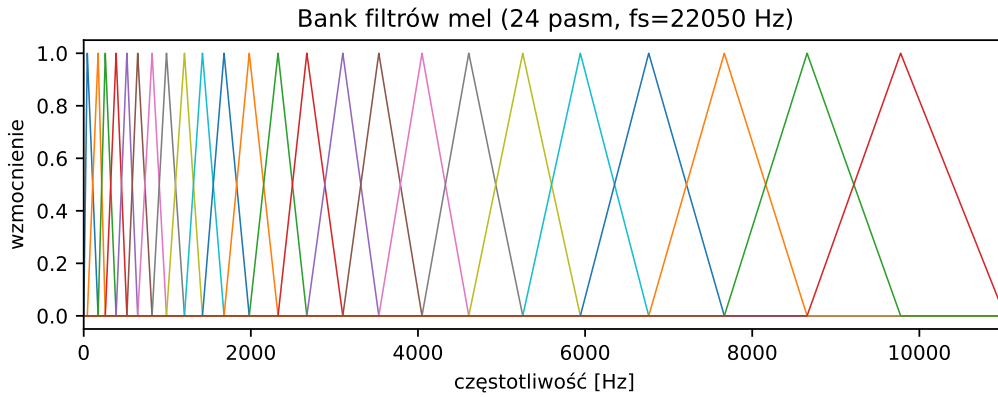


Rys. 2: Sygnał słowa „kot” mówcy 02 przed (lewa kolumna, po samej normalizacji) i po (prawa, po normalizacji i trim’ie ciszy). Każdy z plików tracił po normalizacji ok. 30% próbek z początku i końca.

pasmach mel powstaje przez przemnożenie:

$$E_{\text{mel}}[i] = \sum_{k=0}^{L/2} H_i[k] \cdot |X[k]|^2, \quad i = 0, \dots, 23.$$

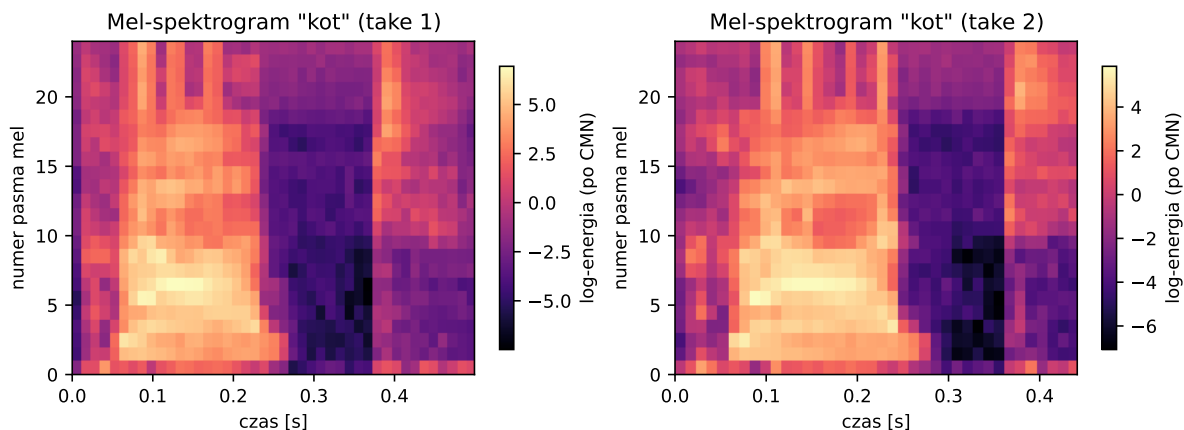
Na koniec bierzemy logarytm: $\phi[i] = \log(E_{\text{mel}}[i] + 10^{-10})$, żeby dynamika cech była zbliżona do percepcji głośności.



Rys. 3: Bank filtrów mel używany do parametryzacji: 24 trójkątne funkcje wagowe rozłożone na skali percepcyjnej dla $f_s = 22,05$ kHz. Pasma niskoczęstotliwościowe są wąskie i gęste, wysokoczęstotliwościowe – szerokie i rzadkie.

Cepstral mean normalization (CMN). Po wyciągnięciu wektorów $\phi_1, \phi_2, \dots, \phi_T$ od ramki do ramki, odejmujemy od nich wektor średni: $\phi'_t = \phi_t - \bar{\phi}$. To kompensuje stały offset addytywny w skali log-energii, który wynika ze stałej charakterystyki kanału (mikrofonu, pomieszczenia).

Wynikiem parametryzacji jest zatem macierz cech o kształcie $T \times 24$, gdzie T to liczba ramek (zwykle 30–80 dla pojedynczego słowa). Rys. 4 pokazuje takie spektrogramy dla dwóch realizacji słowa „kot”.



Rys. 4: Mel-spektrogramy dwóch realizacji słowa „kot” mówcy 02. Wyraźnie widać tę samą strukturę: krótka część szumowa (eksplozja /k/) na początku, dłuższa o silnej energii w niższych pasmach (samogłoska /o/), wygaszanie pod koniec (/t/). Realizacje mają jednak różne długości – 43 i 38 ramek.

6 Dynamic Time Warping

DTW to klasyczna technika wyrównywania dwóch sekwencji o różnej długości, oparta na programowaniu dynamicznym. Mamy dwie macierze cech $X \in \mathbb{R}^{N \times d}$ i $Y \in \mathbb{R}^{M \times d}$. Najpierw budujemy *macierz kosztów lokalnych* $C[n, m]$ – u nas po prostu odległość euklidesowa pomiędzy wektorami:

$$C[n, m] = \sqrt{\sum_{i=1}^d (X[n, i] - Y[m, i])^2}.$$

Następnie liczymy *macierz kosztów skumulowanych* D z rekurencji

$$D[n, m] = C[n, m] + \min(D[n-1, m-1], D[n-1, m], D[n, m-1]),$$

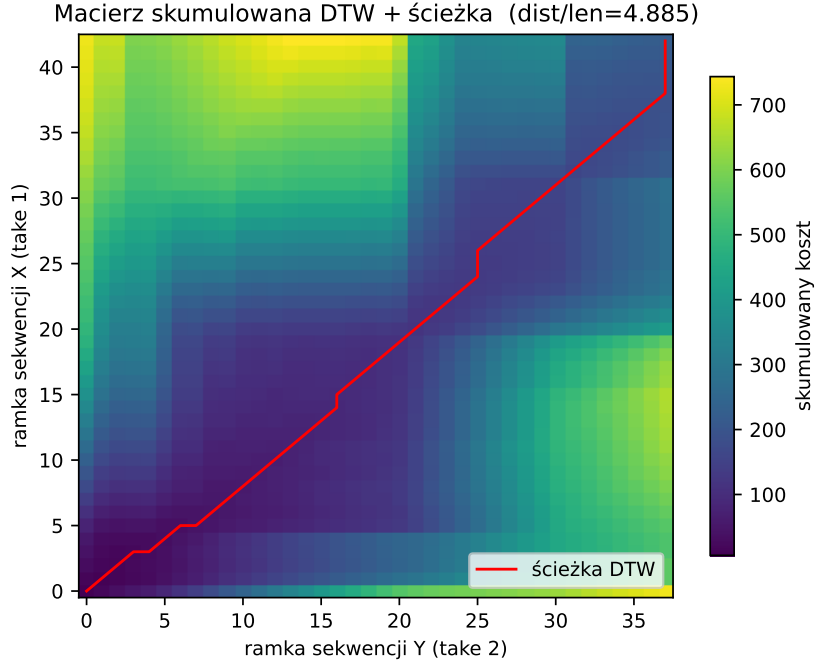
inicjując $D[0, 0] = C[0, 0]$, oraz krawędzie kumulatywnie z pierwszej kolumny i wiersza. Wynikową odległością DTW jest $D[N-1, M-1]$. Ścieżkę dopasowania $(n_0, m_0), (n_1, m_1), \dots$ odtwarzamy idąc wstecz od prawego górnego rogu, w każdym kroku wybierając poprzednika o najmniejszym D .

Ograniczenie Sakoe-Chiba. Bez żadnych ograniczeń DTW dopuszcza skrajnie niesymetryczne dopasowania, w których cała sekwencja X jest „przyklejona” do jednego fragmentu Y . Dla mowy to fizycznie nie ma sensu, realizacje tego samego słowa różnią się długością raczej w granicach $\pm 30\%$, a nie kilkukrotnie. Ograniczenie Sakoe-Chiba ogranicza dozwoloną odległość $|n - m|$ od głównej przekątnej. U nas używamy $\text{band_ratio} = 0.1$, czyli pas o szerokości $\pm 10\%$ dłuższej z sekwencji. Ubocznym efektem jest $5\text{--}10\times$ szybsze liczenie, praktycznie cała pętla w środku i tak idzie po liniowej liczbie elementów na wiersz, a nie po pełnym $N \times M$.

Normalizacja długością ścieżki. Surowa odległość DTW rośnie liniowo z długością ścieżki, a ta z kolei zależy od długości obu sekwencji. Bez kompensacji krótsze nagrania referencyjne mają systematycznie mniejszą sumę kosztów i k-NN łapie się na to. Dzielimy więc odległość DTW przez długość ścieżki dopasowania: $\tilde{D} = D[N-1, M-1]/|\text{path}|$. Na Rys. 5 widać tę ścieżkę narysowaną na macierzy D .

7 Klasyfikator

Klasyfikator to standardowy k-NN po znormalizowanej odległości DTW. Dla zapytania q liczymy odległość do każdego wzorca z bazy referencyjnej, sortujemy rosnąco, bierzemy k najbliższych.



Rys. 5: Macierz skumulowana DTW i optymalna ścieżka dopasowania (czerwona linia) pomiędzy dwoma realizacjami słowa „kot”. Wyraźna „skośność” ścieżki obrazuje rozciąganie pierwszej sekwencji względem drugiej. Wartość znormalizowana $\tilde{D} = 4,885$.

Etykietę wyciągamy głosowaniem ważonym odwrotnością odległości:

$$\text{score}(c) = \sum_{r \in \text{topK}, \text{label}(r)=c} \frac{1}{\tilde{D}(q, r) + \varepsilon}.$$

Zwracamy klasę o najwyższym **score**. Dla $k = 1$ ten schemat sprowadza się do zwykłego **argmin**, dla większego k wzorce dalsze ważą mniej. W ewaluacji niżej używamy $k = 1$, przy $k = 3$ wyniki są bardzo podobne (± 1 pp), a $k = 1$ jest najprostszy do interpretacji.

8 Wyniki

Ewaluację przeprowadziliśmy w schemacie *split per-mówca*: dla każdej pary (mówca, słowo) najwcześniejsze *take* trafia do bazy referencyjnej, a najpóźniejsze do zbioru testowego. To 649 wzorców referencyjnych i 625 zapytań. Pełna runtime ewaluacji w wątku z multiprocessing’iem, ok. 6 minut.

Tab. 1: Ogólne wyniki klasyfikacji.

Zadanie	poprawnych	dokładność
Rozpoznawanie słów	538/625	86,08%
Identyfikacja mówcy	492/625	78,72%

Z 17 mówców i 70 rozpoznawanych słów losowy klasyfikator dostałby $\sim 5,9\%$ dla mówcy i $\sim 1,4\%$ dla słowa, więc oba zadania są wykonywane znacząco lepiej niż przypadek. Tabela 2 pokazuje rozkład wyników na poszczególnych mówców.

Widać dwie skrajności: speaker 28 wypada perfekcyjnie (100% na słowach, 97% na mówcy), natomiast speaker 10 jest najsłabszym ogniwem (44% na słowach).

Tab. 2: Dokładność per-mówca. Każdy mówca ma w teście ~ 38 nagrań (po jednym dla każdego z jego słów).

Mówca	słowa	%	Mówca	słowa	%
1	27/38	71	10	14/32	44
2	33/38	87	13	30/38	79
3	36/38	95	14	36/38	95
4	32/33	97	22	29/35	83
5	37/38	97	23	35/38	92
6	36/38	95	24	31/38	82
7	27/37	73	25	30/33	91
8	33/38	87	28	38/38	100
9	34/37	92			

Tab. 3: Najczęściej mylone pary słów. Wiele z nich to fonetyczni bliźniacy lub słowa bezsensowne („aba” / „abe” / „oga” - mówcy traktują je jako jedno słowo).

prawda	przewidziano	#	prawda	przewidziano	#
aba	abe	6	pies	piec	2
abe	oga	3	szesc	szeya	2
gyr	zid	2	jeden	dwa	2
dzwiek	dziewiec	1	kwiat	jajko	1

Najlepsze i najgorsze słowa. Pełne poprawki (100%) zanotowaliśmy m.in. dla słów *trzy*, *benzyna*, *chrząszcz*, *jajko*, *odrutowiec*, *samochód*. To w przeważającej części słowa wielosylabowe o jasno odróżnialnej strukturze fonetycznej. Najgorzej szły krótkie pseudosłowa typu *aba*, *gyr*, *zid*, one są w bazie pierwotnie jako filler do testów, niemal wszyscy mówcy wymawiali je z innym akcentem i z minimalną ekspresją.

9 Wnioski

Najprostszy klasyfikator typu k-NN po DTW, po starannym potraktowaniu wstępnym sygnału (trim ciszy, pre-emfaza, CMN) i parametryzacją mel-FFT, osiąga w naszej bazie 86% skuteczności na rozpoznawaniu słów i 79% na identyfikacji mówcy. Wynik jest stabilny – różnice ± 1 pp pomiędzy $k = 1$ i $k = 3$, a głównymi źródłami błędów są fonetycznie zbliżone słowa krótkie (zwykle pseudosłowa typu *aba/abe*) oraz pojedynczy mówca o szczególnie nierównych nagraniach (*speaker 10*).

Najważniejsze lekcje z implementacji są niezwiązane bezpośrednio z algorytmem DTW:

- zgodność cech między bazą referencyjną a zapytaniem jest niepodległa dyskusji – jednowymiarowe RMS w jednym miejscu, 24-wymiarowy mel w drugim i klasyfikator po prostu nie działa;
- trim ciszy i pre-emfaza dają więcej niż żonglowanie hiperparametrami DTW;
- ostatecznie 90% jakości projektu to dyscyplina w obsłudze danych: kanonizacja nazw plików, jednolite cechy, wspólny split treningowo-testowy.